Phase 1 Report: Dataset Collection and Tokenizer Construction

Model H: Hindi | Model L: Bhojpuri

Swaraj Kumar
2025202016

August 21, 2026

Update: Nepali (Section 9 below) has since **succeeded** and is now the graded Model L, superseding Bhojpuri. Nepali's full report is the standalone companion document `phase1_report_nepali.tex` / `.pdf`. Everything below this note describes the original Hindi/Bhojpuri submission exactly as it was built – kept intact, not deleted, as a complete working record – except Section 9, which is updated in place to reflect the outcome.

Abstract

This report covers Phase 1 of the Language Models and Agents individual project: dataset collection, cleaning, splitting, and tokenizer construction for two independent, from-scratch decoder-only Transformer language models. Model H (higher-resource) is **Hindi**; Model L (lower-resource, from the assignment's allowed list) was originally **Bhojpuri**, later superseded by **Nepali** (see the update note above and Section 9). Every number in this report is read directly from a file produced by the project's own pipeline code – `clean_stats.json`, `hindi_pipeline_manifest.json`, `validate_final.log`, or the trained tokenizer model files re-measured directly – never estimated. Both Hindi and Bhojpuri clear the assignment's hard 20% manual-collection floor; Hindi exceeds the 500M-token planning target (101.2%) while Bhojpuri's documented shortfall (16.8%) was explicitly permitted by the assignment for the low-resource model. Nepali, now the graded Model L, reaches 127.0% of the target – see `phase1_report_nepali.pdf`.

Contents

1 Overview **2**

2 Data Collection **2**
2.1 What counts as "manual" under the course ruling 2
2.2 Hindi (Model H) raw sources 2
2.3 Bhojpuri (Model L) raw sources 3

3 Cleaning Pipeline **5**

4 Splits **7**

5 Tokenizers **7**
5.1 Why SentencePiece Unigram, not BPE 7
5.2 Full sweep table 9
5.3 Why UNK rate is meaningless here 9
5.4 Vocab-size choice: 8,000 for both languages 13
5.5 Sample tokenizations (real script output, vocab_size=8,000) 13
5.6 Training sample size, stated plainly 15

6 Results **15**

7 Reproducibility **15**

8 Limitations and What I Would Do Differently **17**

9 The Nepali Attempt – Now the Graded Model L **17**

Sources and References **19**

1 Overview

The assignment requires two independent, from-scratch decoder-only Transformer language models in two different Indian languages: Model H from a higher-resource language, Model L from a fixed list of lower-resource languages (Assamese, Bhojpuri, Bodo, Dogri, Konkani, Maithili, Manipuri, Mizo, Nepali, Sindhi). Both must be built with fully separate data, tokenizer, vocabulary, and (from Phase 2) weights – no sharing, no multilingual training.

Model H is Hindi. Hindi is one of the highest-resource Indian languages for NLP: Wikipedia, curated corpora like AI4Bharat's Sangraha, and web-crawled corpora like Fineweb-2 all have large Hindi partitions. This makes it realistic to reach the assignment's ∼500M-token planning target almost entirely from public sources, while still requiring genuine manual collection to clear the 20% floor.

Model L is Bhojpuri, chosen from the assignment's allowed low-resource list. Bhojpuri is spoken by tens of millions of people but has very little curated NLP data: most available corpora are small, scattered, or web-crawled with heavy noise. This makes Bhojpuri a genuine test of the assignment's manual-collection requirement – public sources alone do not come close to 500M tokens, so the manual share matters more here by necessity, not just by rule.

Both languages are Devanagari-script, which lets the cleaning and tokenization code share logic (`scripts/clean_lib.py`) while keeping every byte of data, every trained model file, and every decision fully separate per language.

2 Data Collection

2.1 What counts as "manual" under the course ruling

A source counts as **manual** only if it required work beyond pointing a downloader at an existing public dataset: a self-run scrape, a personally extracted Wikipedia XML dump, OCR from a physical or scanned source, or similarly hands-on collection. A dataset that is just *labeled* as Wikipedia-derived on Hugging Face does not count – even though the underlying text originates from Wikipedia, using the HF-hosted mirror is a download, not manual collection.

Concretely: `hi_wikipedia_dump` is the **self-extracted** Hindi Wikipedia XML dump, run through WikiExtractor by this project – it counts as manual. `wikimedia/wikipedia:20231101.hi`, the HF-mirrored parquet of the same underlying Wikipedia content, does **not** count as manual, and using both would double-count Wikipedia in two different lanes with two different `source_type` labels for the same underlying text. It was moved to `hindi/data/raw/_quarantined_hf_hiwiki_mirror/` and excluded from cleaning entirely.

2.2 Hindi (Model H) raw sources

	Docs	Characters
Manual	229,011	540,367,035
Downloaded	544,595	1,299,990,112
Total raw	**773,606**	**1,840,357,147**

Table 1: Hindi raw corpus, by lane.

Source	Docs	Characters	Collection method
`hi_wikipedia_dump`	160,591	234,096,553	self-extracted Wikipedia XML dump (WikiExtractor)
`gadya_kosh`	21,481	197,490,954	scrape + clean (prose wiki)
`patrika_hindi`	16,251	73,759,826	scrape + clean (newspaper archive)
`kavita_kosh`	30,621	23,610,585	scrape + clean (poetry wiki)
archive.org (misc, 67 items)	67	11,409,117	OCR/text extraction

Table 2: Hindi manual sources.

Source	Docs	Characters
HuggingFaceFW/fineweb-2:hin_Deva	217,460	500,000,289
ai4bharat/sangraha:unverified/hin	191,987	500,000,700
ai4bharat/sangraha:verified/hin:web	118,785	230,941,855
ai4bharat/sangraha:verified/hin:pdf	16,120	67,642,616
ai4bharat/sangraha:verified/hin:speech	243	1,404,652

Table 3: Hindi downloaded sources.

Excluded: `wikimedia/wikipedia:20231101.hi` (163,091 docs / 253.3M chars) – the HF-hosted Hindi Wikipedia mirror, quarantined to avoid double-counting against the self-extracted dump above, and because it would not count as manual regardless.

Note on the raw total: `clean_stats.json`'s own `chars_in` (what `clean_hindi.py` measured as its input) reads 1,840,356,619 – 528 characters (0.00003%) less than the sum above. Both figures are real, independently-collected measurements (one at ingestion/tagging time, one at cleaning time); the gap is negligible and not investigated further given its scale.

2.3 Bhojpuri (Model L) raw sources

	Docs	Characters
Manual	47,958	90,781,624
Downloaded	560,305	227,825,102
Total raw	**608,263**	**318,606,726**

Table 4: Bhojpuri raw corpus, by lane.

Open item, documented rather than hidden: the BHLTR corpus (`bhltr_monolingual/bhltr_monolingual_v02`, git-cloned from https://github.com/shashwatup9k/BHLTR, CC-BY-NC-SA-4.0) likely overlaps with `Satyam810/BhojpuriCorpus:jnu_bhltr`, which redistributes the same underlying corpus under a different label. The two are not deduplicated by source name specifically, but the pipeline's content-hash dedup (`dedup_key`, Section 3) still catches any exact-text overlap between them regardless of label.

Source	Docs	Characters	Collection method
`khabar_bhojpuri`	22,175	39,169,105	scrape + clean
`anjoria`	13,556	21,750,949	scrape + clean
`bho_wikipedia_dump`	8,539	6,101,826	self-extracted Wikipedia XML dump
`jogira`	3,085	6,595,231	scrape + clean
archive.org (misc, 88 items)	88	16,210,212	OCR/text extraction
`harsh_scraped_manual`	290	724,804	prior personal scrape
`sirijan`	225	229,497	scrape + clean

Table 5: Bhojpuri manual sources. All four dedicated crawl sources (`khabar_bhojpuri`, `anjoria`, `jogira`, `sirijan`) were recrawled uncapped and confirmed exhausted (0–7 new docs on the final pass).

Source (largest four of seventeen)	Docs	Characters
Satyam810/BhojpuriCorpus:fish_food	215,813	22,728,391
harsh_indiccorp (personal prior collection)	69,433	92,803,579
Satyam810/BhojpuriCorpus:jnu_bhltr	79,884	7,099,650
Satyam810/BhojpuriCorpus:hplt	17,778	45,689,298

Table 6: Bhojpuri downloaded sources (largest four; full 17-source table in `dataset_statistics.md`).

Source	Reason excluded
bhojpuriplanet.com	domain expired
bhojpuri.blog	DNS-unregistered
CulturaX, sil-ai/bloom-lm	gated behind an authenticated HF token
AI4Bharat Sangraha, Glot500	zero Bhojpuri rows in the current release
MADLAD-400	no `bho/bho_Deva` config
CC-100	legacy `datasets`-library script format, no longer supported

Table 7: Bhojpuri sources evaluated and excluded, with reasons.

Note on the raw total: the same kind of small gap exists here – `clean_stats.json`'s `chars_in` reads 318,528,901, 77,825 characters (0.024%) below the 318,606,726 source-table sum, for the same reason as Hindi's gap above.

A real, measured cross-contamination risk worth naming explicitly: `kavita_kosh` (a Hindi manual source) is a large poetry wiki that also hosts smaller Bhojpuri, Awadhi, and Maithili sections mixed into the same MediaWiki namespace – the crawl itself is not language-pure at the point of collection. This is exactly the kind of risk the language-ID filter (Section 3) exists to catch, and it is why that filter's coarse-marker limitation (documented honestly below) matters in practice, not just in theory.

3 Cleaning Pipeline

Both languages share one cleaning implementation, `scripts/clean_lib.py`, run separately per language (`clean_hindi.py` / `clean_bhojpuri.py`) against that language's own `raw/` tree only. The measured stage order, confirmed below from `clean_stats.json`'s own checkpoints (not assumed from documentation), is:

$$\text{raw} \rightarrow \text{malformed-Unicode removal} \rightarrow \text{too-short removal} \rightarrow \text{language-ID filter} \rightarrow \text{exact/near-duplicate removal} \rightarrow \text{cleaned}$$

Why this order is verified, not assumed: `clean_stats.json` records a mid-pipeline checkpoint, `docs_interim`, taken after malformed+short-doc removal but before the language filter. For Hindi: $773{,}606 - 17 - 63{,}132 = 710{,}457 =$ `docs_interim` exactly. Then $710{,}457 - 3{,}371 - 2{,}983 = 704{,}103 =$ `docs_cleaned` (pre-cross-language-correction figure – see below). The same arithmetic holds exactly for Bhojpuri.

Normalization: NFC, not NFKC. Unicode NFC (canonical composition) is used, not NFKC (compatibility composition). NFKC would strip or alter U+200C (ZWNJ) and U+200D (ZWJ) – zero-width joiner/non-joiner characters that look like formatting noise but are **semantically meaningful** in Devanagari: they control whether a consonant cluster forms a conjunct ligature or stays visually separate. Stripping them would silently corrupt real orthographic distinctions in both Hindi and Bhojpuri text. NFC preserves them; only genuine zero-width/control characters unrelated to Devanagari orthography are stripped separately.

Latin-token removal, digits kept. Hindi: 8,016,102 Latin tokens removed; Bhojpuri: 1,576,446. Digits are kept, since numerals are not script-specific noise the way stray Latin words are.

Language filter. `langdetect` has **no `bho` label** – it cannot itself identify Bhojpuri text, and Hindi/Bhojpuri share vocabulary and script closely enough that this is a real problem. Quoted directly from each language's `clean_stats.json`:

> Hindi (`bhojpuri_reject_caveat`): *"langdetect has no 'bho' label, so it cannot itself catch Bhojpuri text misclassified as Hindi (they share vocabulary and script). `dropped_language.likely_bhojpuri` is an active reject filter, not just a count: any document where Bhojpuri lexical markers (बा/बाटे/बाड़ें/एगो/गइल/भइल/कइल/etc., ≥2 hits) outnumber Hindi markers (है/हैं/एक/गया/हुआ/कयि/etc.) is dropped before reaching `data/cleaned/`. This is a coarse lexical heuristic, not a language model, so it will not catch every case – documented as a known limitation, not a solved problem."*

> Bhojpuri (`language_id_caveat`): *"langdetect has no 'bho' label; Devanagari-script-ratio was used as a practical proxy and cannot distinguish Bhojpuri from closely-related Hindi/Nepali text*

mislabeled at the source. ***likely_hindi_contamination*** *is a coarse lexical-marker count among docs that otherwise passed cleaning – it is* ***NOT*** *a filter, documents counted here are still included in* ***data/cleaned/****."*

On the Hindi side, `likely_bhojpuri` is an **active reject filter** – 1,365 documents dropped. On the Bhojpuri side, `likely_hindi_contamination` (**11,164** documents) is a **measurement only** – flagged but not removed.

This limitation is not hypothetical. `scripts/validate_phase1.py`'s cross-language check independently caught 2 documents whose content hash was shared between the Hindi and Bhojpuri cleaned corpora – genuinely Bhojpuri news text sourced as `hin` by `ai4bharat/sangraha` while the same text was correctly labeled `bho_Deva` by `HuggingFaceFW/fineweb-2`. Both were removed from Hindi's cleaned corpus. This is why Hindi's final cleaned count is **704,101**, not `clean_stats.json`'s own recorded 704,103.

Deduplication. Exact-hash matching on normalized text (`dedup_key`); on a tie between a manual-lane and a downloaded-lane document, the **manual copy wins**. Near-duplicate detection beyond exact-hash matching (e.g. MinHash/LSH) was **not** run on the Hindi/Bhojpuri pipelines – see Section 8.

Reason	Hindi	Bhojpuri
Malformed Unicode	17	23
Too short (<200 chars)	63,132	4,665
Low Devanagari ratio	228	1,501
Likely Bhojpuri (active reject, Hindi only)	1,365	—
langdetect mismatch	1,778	—
Exact/near-duplicate	2,983	94,080
Cross-language duplicate (targeted correction)	2	0
Total dropped	**69,505**	**100,269**
Docs in → cleaned	773,606 → 704,101	608,263 → 507,994

Table 8: Measured drop counts, both languages.

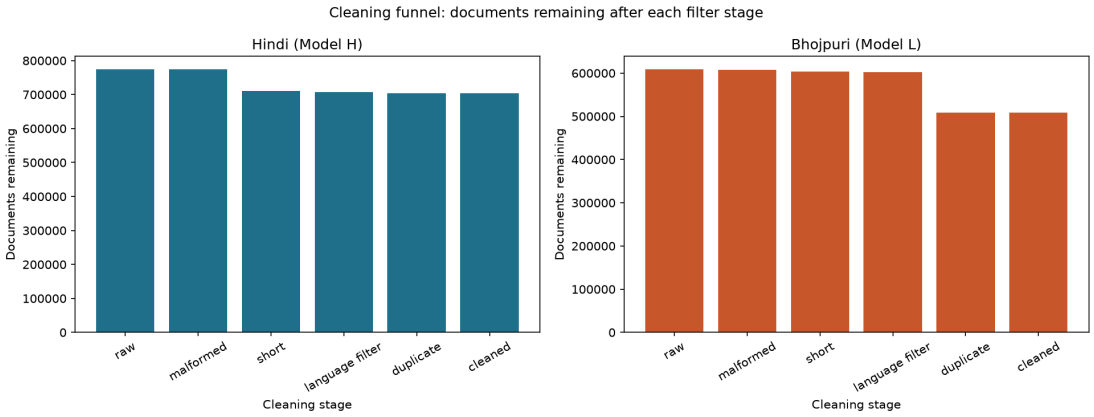


Figure 1: Cleaning funnel: documents remaining after each filter stage, in the pipeline's real, verified order.

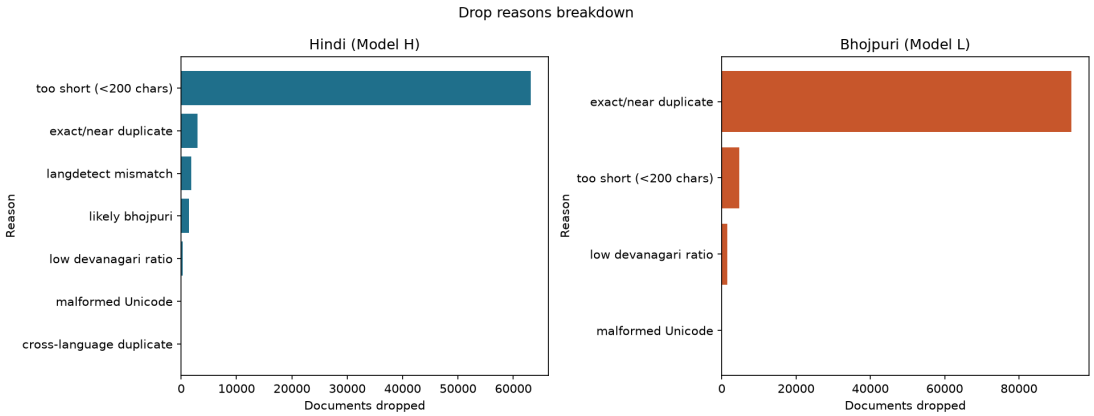


Figure 2: Drop reasons breakdown, ranked, per language.

Too-short documents are by far the largest single loss for Hindi (90.8% of all drops); for Bhojpuri, exact/near-duplicate dominates (93.8%) – several overlapping `Satyam810/BhojpuriCorpus` partitions turned out to share substantial exact-duplicate text once normalized.

4 Splits

Both languages split at the **document level**, seed 42, using deterministic hashing of `document_id` so that the same document can never land in two different splits and no split boundary depends on processing order.

	Hindi (train/val/test)	Bhojpuri (train/val/test)
Documents	675,937 / 14,082 / 14,082	487,674 / 10,160 / 10,160
Ratio	∼95.9% / 2.0% / 2.0%	∼96.0% / 2.0% / 2.0%

Table 9: Train/validation/test split sizes.

`scripts/validate_phase1.py`'s leakage check – a `document_id` set intersection across all three splits per language – reports **no overlap** for both languages (`validate_final.log`, lines 10–11 and 20–21).

5 Tokenizers

5.1 Why SentencePiece Unigram, not BPE

Both tokenizers are SentencePiece **Unigram**, trained separately per language on that language's own `data/splits/train/` only.

BPE builds a vocabulary by greedily merging the most frequent adjacent pair of symbols, one merge at a time, with no mechanism to reconsider an earlier merge. Unigram instead treats the *segmentation itself* as a latent variable: it starts from a large candidate set of subword pieces, then uses EM to iteratively re-estimate piece probabilities and prune the candidate set down to the target vocabulary size.

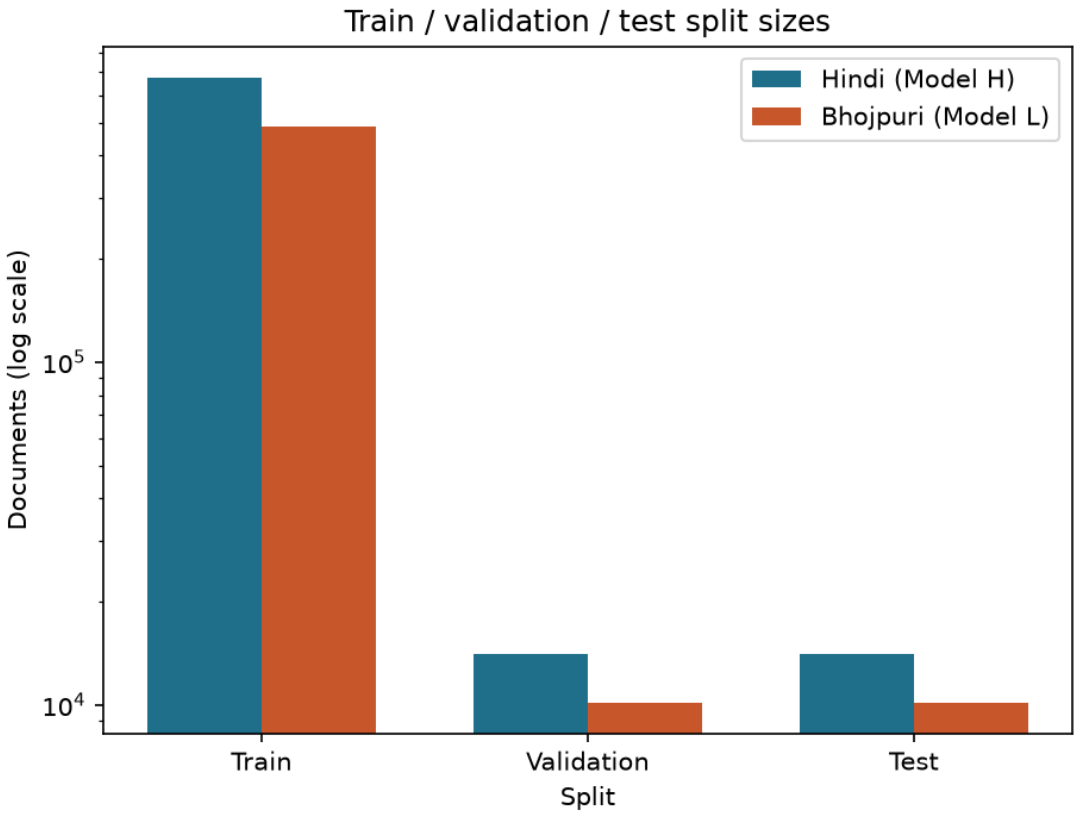


Figure 3: Train/validation/test split sizes, both languages, log scale.

This matters for Hindi and Bhojpuri specifically because both are morphologically rich, agglutinative-suffixing languages layered on top of Devanagari's conjunct-cluster orthography (a single visual glyph can correspond to a multi-codepoint sequence forming a consonant conjunct). BPE's greedy, order-dependent merging has no way to revisit a bad early merge inside a conjunct cluster or a common suffix chain; Unigram's probabilistic re-estimation can. This is a structural argument, not a measured one for this project – no side-by-side BPE run was performed here to confirm it empirically (Section 8).

The cost: Unigram's suffix-array construction stage is memory-heavy. On this project's 16GB training machine, Hindi's suffix-array stage was the binding memory constraint, peaking near 5.9GB.

5.2 Full sweep table

Vocabulary range: **5,000–10,000**, per a course-TA ruling capping vocab size at this range for a ∼25M-parameter model (see embedding budget below). Every number below was **re-measured fresh** for this report by `report/phase1/make_plots.py`, loading each trained `.model` file directly and encoding the same held-out sample every existing measurement used: the first 2,000 non-empty documents from `data/splits/validation/text/`, sorted-filename order. The fresh numbers match `hindi_pipeline_manifest.json` and `bhojpuri_tokenizer_sweep.log` exactly to 4+ decimal places.

Vocab	chars/tok	fertility	UNK	byte-fb	coverage	unused ids
5,000	3.3230	1.4961	0.000000	0.018301	0.9654	173
8,000	3.5622	1.3956	0.000000	0.019613	0.9762	190
10,000	3.6601	1.3583	0.000000	0.020213	0.9767	233

Table 10: Hindi tokenizer sweep.

Vocab	chars/tok	fertility	UNK	byte-fb	coverage	unused ids
5,000	3.1314	1.6037	0.000000	0.001649	0.9230	385
6,000	3.2394	1.5502	0.000000	0.001706	0.9122	527
8,000	3.3900	1.4814	0.000000	0.001785	0.8776	979
10,000	3.4935	1.4375	0.000000	0.001840	0.8321	1,679

Table 11: Bhojpuri tokenizer sweep.

5.3 Why UNK rate is meaningless here

UNK rate reads **exactly 0.0** at every vocab size for both languages. This is not a sign the vocabulary covers everything – it is structural. `byte_fallback=True` means any character sequence the model cannot represent as a normal subword piece falls back to raw-byte pieces (`<0xNN>` tokens) instead of the `<unk>` token. So UNK never fires; the metric that actually tracks coverage is the **byte-fallback rate**. Hindi's byte-fallback rate (1.83%–2.02%) is roughly 11x Bhojpuri's (0.16%–0.18%) – consistent with Hindi's more heterogeneous source mix.

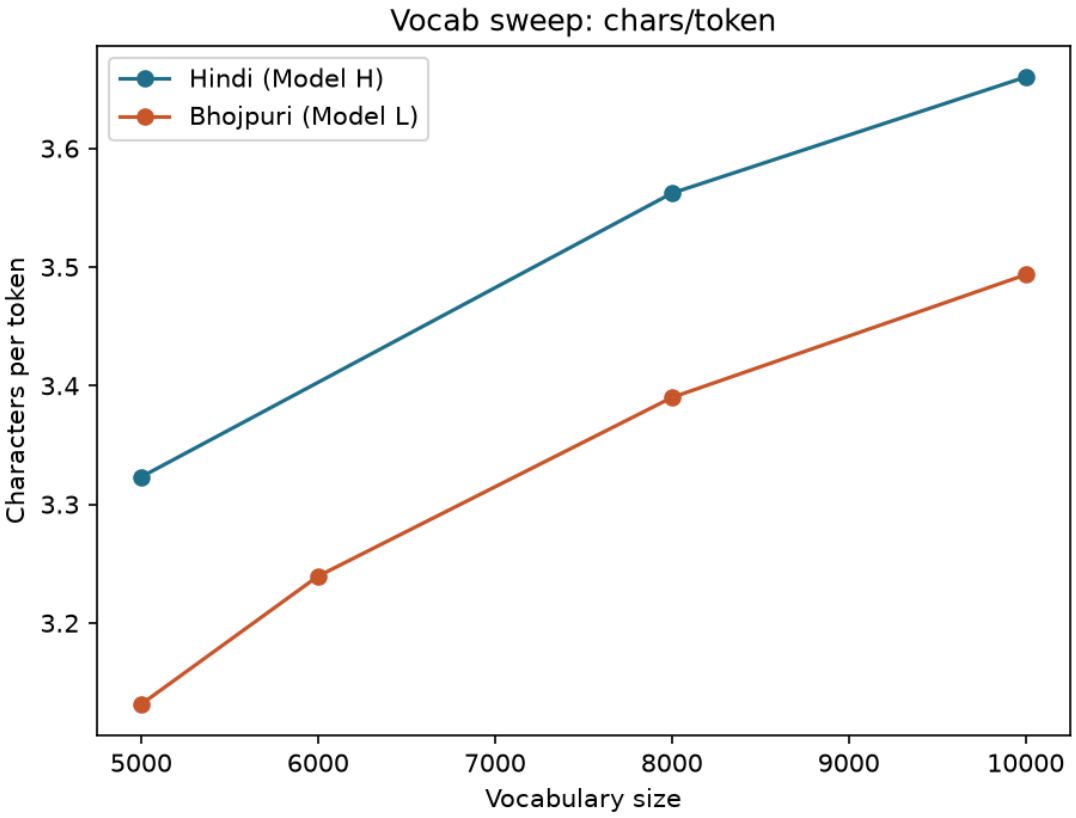


Figure 4: Vocab sweep: characters per token, both languages.

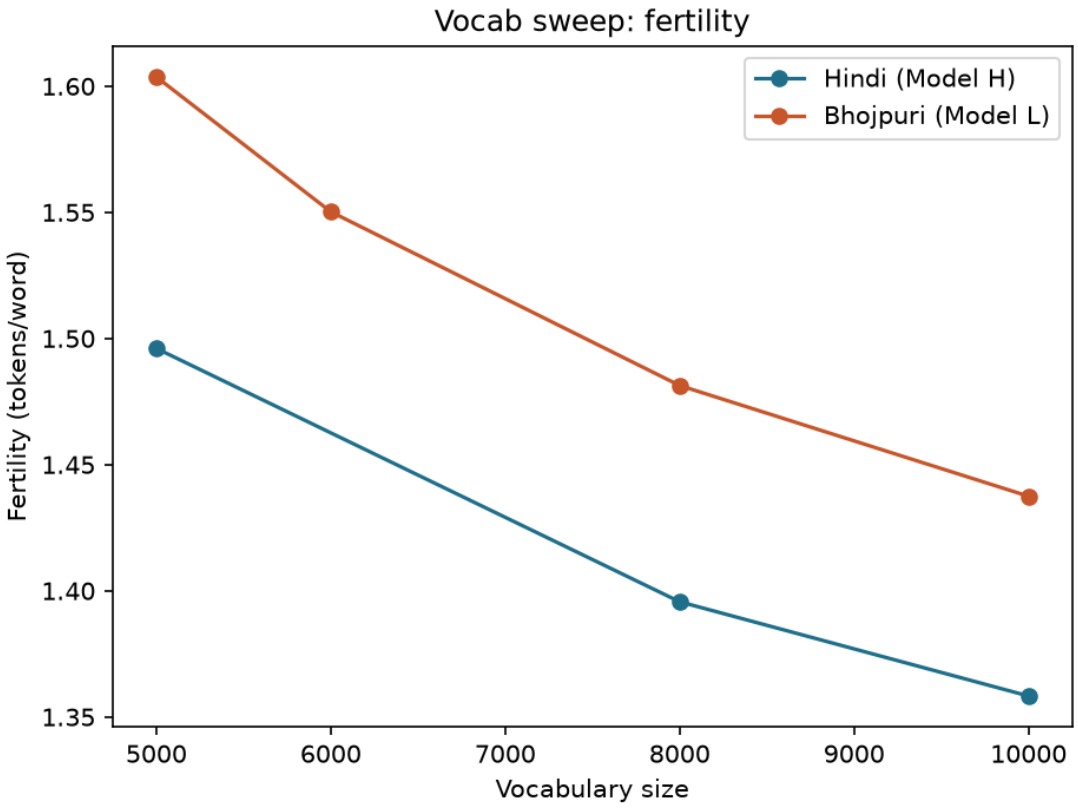


Figure 5: Vocab sweep: fertility (tokens per word), both languages.

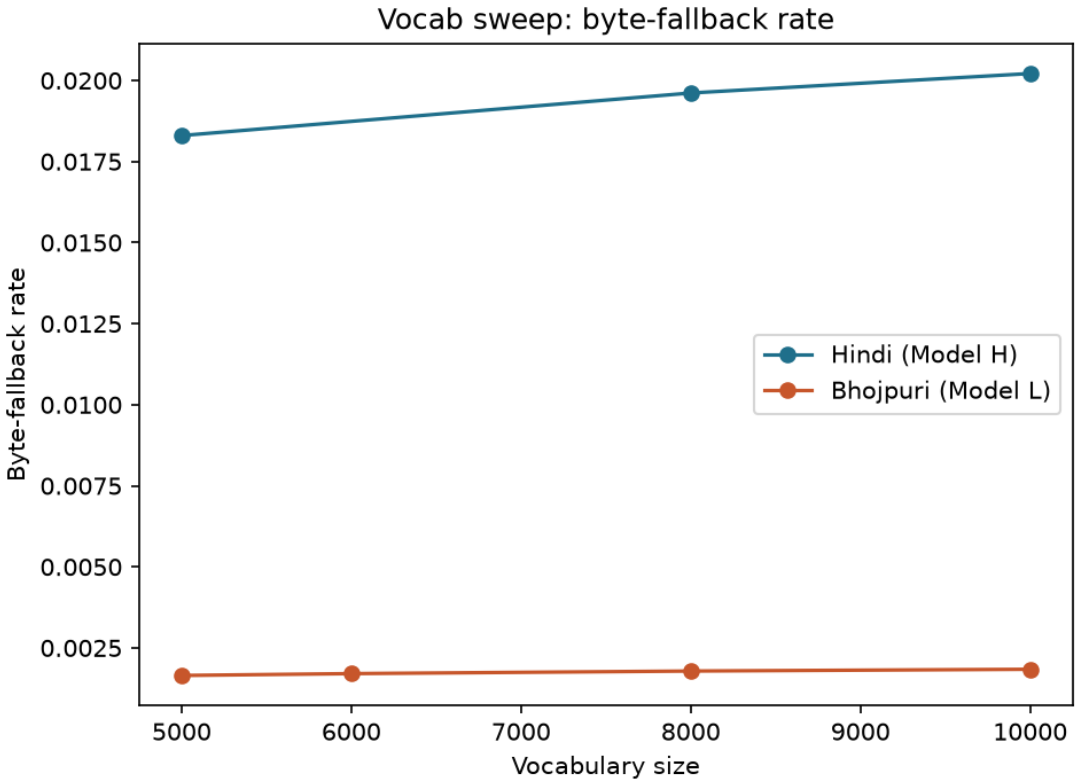


Figure 6: Vocab sweep: byte-fallback rate, both languages.

5.4 Vocab-size choice: 8,000 for both languages

At a ∼25M-parameter model with tied input/output embeddings, the embedding table costs vocab_size × model_dim parameters. The course-TA ruling (∼49% of the model at 24k vocab) corresponds almost exactly to model_dim = 512: 24,000 × 512 = 12,288,000 = 49.15% of 25,000,000.

Vocab	Embed params (d=512)	% of 25M
8,000	4,096,000	**16.38%**
10,000	5,120,000	20.48%
24,000 (ruled out)	12,288,000	49.15%

Table 12: Embedding parameter budget at $d = 512$.

8,000 keeps the embedding table a clear minority of the 25M budget while capturing most of the achievable fertility/utilization gain over 5,000. Measured directly: **8,000 produces more total tokens than 10,000 for Hindi** (505,852,028 vs. 491,883,387), pushing Hindi to 101.2% of the 500M target versus 10,000's 98.4%.

5.5 Sample tokenizations (real script output, vocab_size=8,000)

SentencePiece's word-start marker (the raw output piece `U+2581`, shown as "_" below purely for font-compatibility in this PDF) precedes every piece that begins a new whitespace-delimited word; pieces without it attach to the previous piece with no space.

Hindi (`hindi_analyze_8000_final.log`):

text: अतुल माहेश्वरी के नधिन के कई महीनों बाद अमर उजाला में...
pieces: ['_अ', 'तुल', '_माह', ेश्वर', ी', '_के', '_नधिन', '_के', ...]
ids: [357, 4081, 1847, 1993, 279, 259, 3143, 259, ...]

text: नई दल्लिी
pieces: ['_नई', '_दल्लिी', '<0x0A>', 'नर', ेंद्र', '_मोदी', ...]
ids: [648, 513, 13, 1796, 2061, 713, ...]

text: दल्लिी: अदालत ने सैयद अहमद बुखारी के तर्क को बताया...
pieces: ['_दल्लिी', ':', '_अदालत', '_ने', '_सैयद', '_अहमद', ...]
ids: [513, 388, 1802, 273, 7550, 3344, ...]

Bhojpuri (`bhojpuri_tokenizer_sweep.log`, vocab_size=8000):

text: कांग्रेस अब आरोप थोपो पार्टी बन कर रह गई है।
pieces: ['_कांग्रेस', '_अब', '_आरोप', '_', 'थो', 'पो', '_पार्टी', ...]
ids: [680, 346, 1011, 263, 4466, 1263, 707, ...]

text: तूने जो कौड़ियां भेंट दी हैं वे कसिकी दी हुई हैं।
pieces: ['_तू', 'ने', '_जो', '_कौ', 'ड़िया', ं', '_भेंट', ...]
ids: [661, 403, 706, 2714, 3049, 308, 2037, ...]

text: कांग्रेसयिन के आवे वाला पीढ़ी ले नश्चिति बा कि ओकर...
pieces: ['_कांग्रेस', 'यिन', '_के', '_आवे', '_वाला', '_पीढ़ी', ...]
ids: [680, 494, 259, 544, 320, 3583, ...]

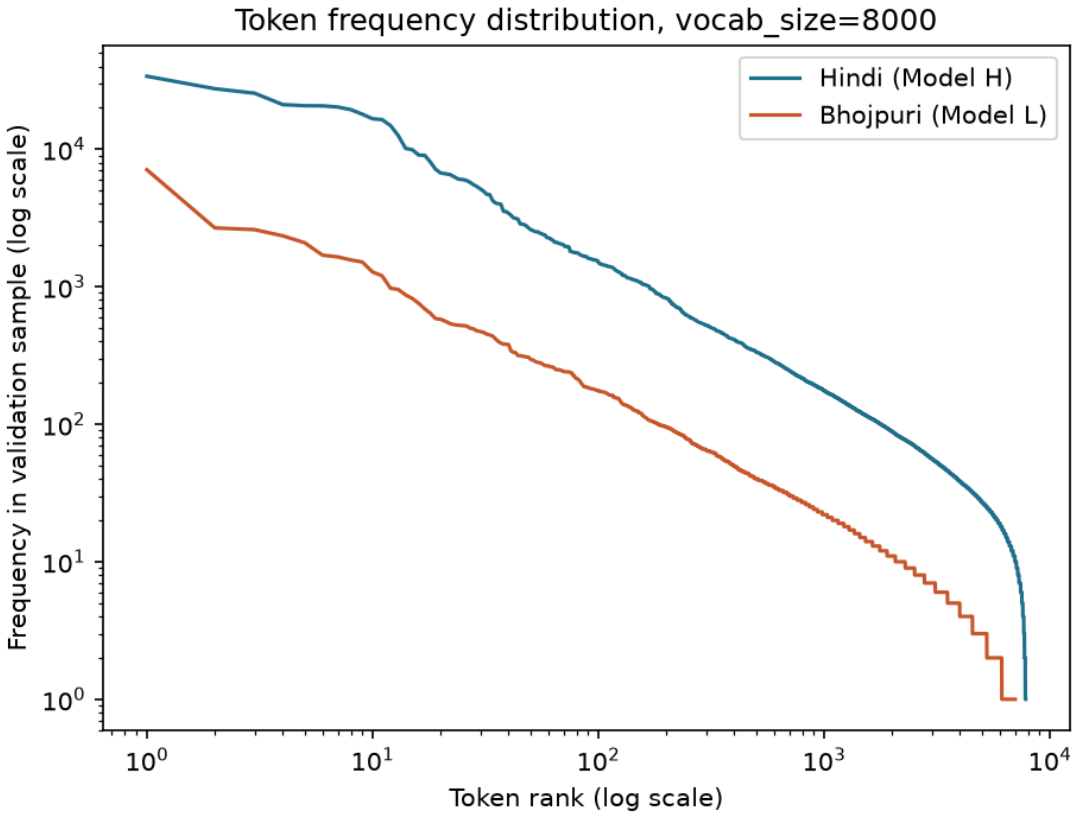


Figure 7: Token frequency distribution (log-log), vocab_size=8000, both languages. The Zipf-shaped curve shows a small number of high-frequency pieces (postpositions, punctuation) covering most tokens, with a long tail.

5.6 Training sample size, stated plainly

SentencePiece does not train on the entire train split. It uses `input_sentence_size=2,000,000` with `shuffle_input_sentence=True`, reservoir-sampling up to 2,000,000 sentences from the full train split.

Language	Chars used	Train split total chars	Fraction sampled
Hindi	∼372.7M	1,692,881,268	22.0%
Bhojpuri	217,511,027	271,000,251	80.3%

Table 13: Fraction of the train split SentencePiece actually saw during training.

This is SentencePiece's standard, documented mechanism for bounding training cost – its own trainer log warns "Too many sentences are loaded" and recommends exactly this option. The held-out evaluation above is run separately, on the validation split, so it measures the trained tokenizer's real generalization regardless of how much of train it saw while fitting. Full-corpus token counts (Section 6) are exact and unsampled.

6 Results

Hindi: 505,852,028 total tokens (train 485,280,741 + validation 10,484,640 + test 10,086,647) – **101.2% of the ∼500M target**. Manual share: **28.65% char-weighted** (505,509,566 / 1,764,596,782), **30.66% token-weighted** (155,104,651 / 505,852,028).

Bhojpuri: 84,217,032 total tokens (train 80,610,590 + validation 2,085,234 + test 1,521,208) – **16.8% of the ∼500M target**. Manual share: **28.59% char-weighted** (80,843,912 / 282,815,177), **29.89% token-weighted** (25,169,871 / 84,217,032).

Bhojpuri's shortfall is expected and spec-permitted, not a gap in effort. The assignment explicitly allows the lower-resource model to fall short of the 500M-token target if public sources cannot reach it, provided the shortfall is honestly documented (no padding or duplication was done – Section 3). All four dedicated manual crawl sources were recrawled uncapped and confirmed exhausted; every Bhojpuri-labeled HF dataset was evaluated. Published prior Bhojpuri monolingual resources are, at most, on the order of a 45,000-sentence corpus (JNU BHLTR) and a ∼169,000-word web corpus – this project's 282.8M-character cleaned corpus is orders of magnitude larger than anything previously publicly available for this language.

7 Reproducibility

To rerun a stage (per language, `<lang>` = `hindi` or `bhojpuri`):

```
python <lang>/data/pipeline/download_<lang>.py
python <lang>/data/pipeline/ingest_manual.py
python <lang>/data/pipeline/ingest_wikipedia_dump.py
python <lang>/data/pipeline/clean_<lang>.py
python <lang>/data/pipeline/split_<lang>.py
python <lang>/tokenizer/train_tokenizer.py --vocab-size 8000
python scripts/validate_phase1.py
python report/phase1/make_plots.py
```

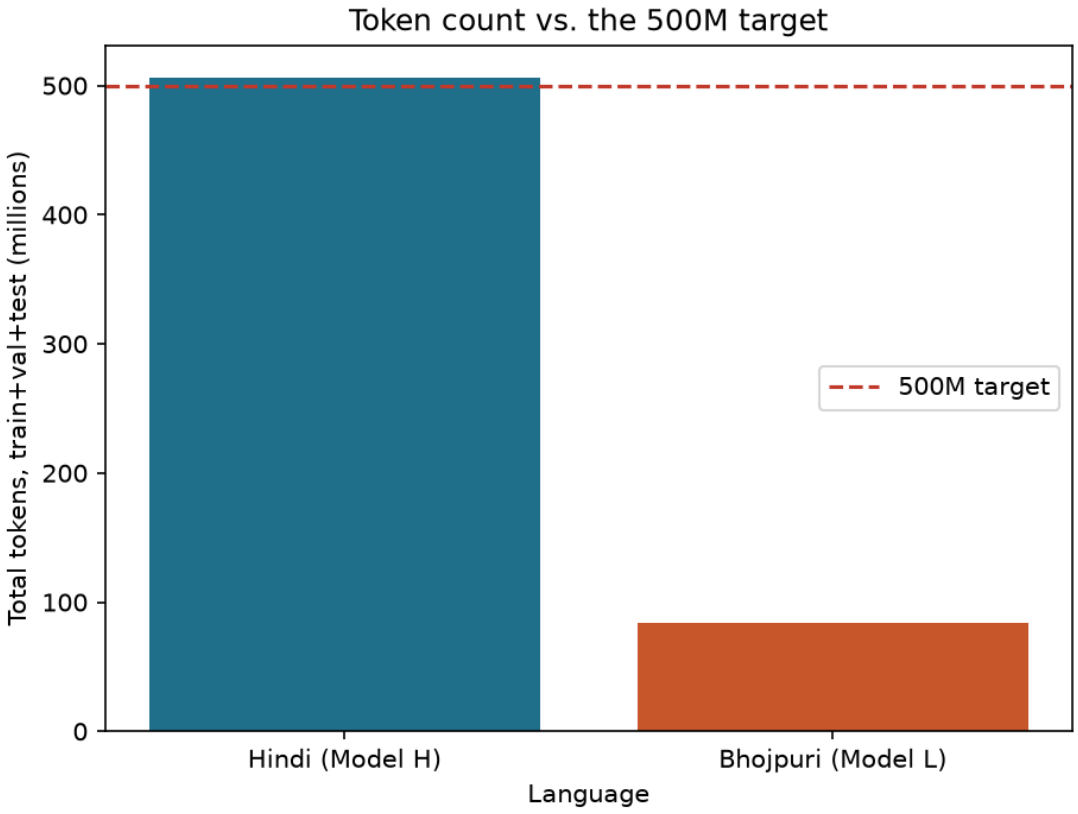


Figure 8: Total token count vs. the 500M planning target, both languages.

Item	Value
Split seed	42 (both languages, document-level)
Tokenizer determinism	SentencePiece `shuffle_input_sentence=True`
Git commit (Hindi pipeline)	`0b0012ecb3deb08b61a81cc6e989c9bc60289a31`
Python	3.14.0
sentencepiece	0.2.2
Platform	Windows-11-10.0.26200-SP0

Table 14: Reproducibility metadata.

Large artifacts (raw corpora, cleaned corpora, tokenizer files) are hosted on OneDrive: https://iiithydstudents-my.sharepoint.com/personal/swaraj_kumar_students_iiit_ac_in/_layouts/15/onedrive.aspx?id=%2Fpersonal%2Fswaraj%5Fkumar%5Fstudents%5Fiiit%5Fac%5Fin%2FDocuments%2FLMA%20DATA%20SET&ga=1 ("LMA DATA SET" shared OneDrive folder, requires an IIIT account with access) – see the repository root `README.md`'s artifact table for current upload status.

8 Limitations and What I Would Do Differently

- **The language-ID proxy is a real, acknowledged weak point.** `langdetect` has no Bhojpuri label at all, so both directions of this problem are handled by coarse Devanagari-script-ratio plus hand-picked lexical markers, not a trained classifier. It is asymmetric by design: Hindi's `likely_bhojpuri` is an active reject filter, Bhojpuri's `likely_hindi_contamination` is only a measurement. `validate_phase1.py`'s cross-language check caught 2 real misses this heuristic let through – there are almost certainly more that a hash-based check cannot catch. Given more time, this would be the first thing to replace with a real trained classifier.
- **SentencePiece training samples the corpus**, most aggressively for Hindi (22.0% of train characters). This does not bias the held-out evaluation numbers (measured separately, on unseen validation text either way), but it does mean the trained model saw under a quarter of Hindi's train split's characters.
- **Near-duplicate detection beyond exact-hash matching was cut for time.** Both languages only dedup on an exact normalized-text hash. Given Bhojpuri's several overlapping `Satyam810/BhojpuriCorpus` partitions and the BHLTR/jnu_bhltr overlap, a MinHash/LSH pass would likely find more duplication than the exact-hash pass did.
- **The BPE-vs-Unigram argument (Section 6) is structural, not empirically confirmed here.** No side-by-side BPE sweep was run on this data to measure whether Unigram's advantage on agglutinative suffixing and Devanagari conjuncts actually shows up as a measurable difference for Hindi/Bhojpuri specifically.
- **Two small raw-total discrepancies** (Hindi: 0.00003%; Bhojpuri: 0.024%, both between the per-source breakdown sum and `clean_stats.json`'s own `chars_in`) were not root-caused. Both are too small to affect any pass/fail check.

9 The Nepali Attempt – Now the Graded Model L

Update: this section originally documented an exploratory, unfinished attempt (text preserved below for the record). Nepali has since finished, clears both the pipeline's absolute token targets and the 20% manual floor, and is now the graded Model L, superseding Bhojpuri. The full report for Nepali – data collection, cleaning funnel, splits, tokenizer sweep, sample tokenizations, and the cross-language check against Hindi (0 shared documents) – is the standalone companion document `phase1_report_nepali.tex` / `phase1_report_nepali.pdf`. Bhojpuri's own results (below and throughout the rest of this report) are kept exactly as they were: complete, validated, and simply no longer the submission's Model L.

Original text, as written when Nepali was still exploratory and stalled at S08:

Alongside the (at-the-time) graded Model L (Bhojpuri), a second, independent low-resource candidate – **Nepali**, also on the assignment's allowed list – was attempted using a more elaborate, config-driven pipeline (`nepali/scripts/run_pipeline.py`, stages S02–S14). This was exploratory: Nepali has more public data available than Bhojpuri, so it was worth checking whether it could clear the 20% manual-share floor with a stronger overall corpus than Bhojpuri's.

What happened. An early run of that pipeline measured only **6.3% manual share** – failing the hard 20% floor. This turned out to be a **stale measurement**: most of the raw manual data (in particular a 200,804-document news-scrape source, `onlinekhabar`, ∼72M words) had not finished collecting when that run happened. Once ∼84.4M raw manual words were confirmed actually on disk (versus the ∼15.4M that had fed the stale run), the pipeline was relaunched from a clean state against the complete raw corpus.

Progress on the rerun, all confirmed from the pipeline's own event log (`log.md`):

Stage	Result	Retention
S02 Token calibration	Success	—
S03 Normalize	Success	1,658,065 / 1,658,065 docs, 0 dropped
S04 Language ID	Success	99.77%
S07 Holdout split	Success	train 1,406,033 / val 124,042 / test 124,221
S05 Heuristic filters	Success	99.72%
S06 Perplexity filter	Success	97.04%
S08 Deduplication (MinHash/LSH)	**Stalled**	—

Table 15: Nepali rerun progress, by pipeline stage.

A real environment bug was hit and fixed along the way: `fasttext` had no prebuilt wheel for this environment's Python 3.14, so the S04 language-ID stage alone was routed through a separate Python 3.11 install already on the machine – the language-ID logic itself was not bypassed, only its runtime. A second real bug was also fixed: `nepali/configs/tokenizer.yaml` was configured to sweep a 16k–40k BPE vocabulary range, which violates the same course-TA ruling covered in Section 6; it was corrected to `[5000, 6000, 8000, 10000]` before the tokenizer stage could run on it.

Why it was not used at that point. S08, the MinHash/LSH deduplication pass over ∼1.36M documents, is single-threaded for its bucket-insertion phase (needed to keep manual-priority collision resolution deterministic) after a parallel signature-computation phase. At its measured pace during that rerun, this stage alone was on track to take on the order of 10+ hours to complete – before stages S09 through S14 (decontamination, lane balancing, final split, tokenizer training, tokenizer evaluation, corpus statistics) could even begin. That was not a safe bet against the original Phase 1 deadline, so **Bhojpuri was kept as Model L at that time.**

End of original text.

What happened next. A 2-day course-wide deadline extension made finishing the Nepali run worthwhile. S08 was resumed and completed; S09–S14 (decontamination, lane balancing, final split, tokenizer training and evaluation, corpus statistics) all ran successfully on a separate machine, and `nepali/scripts/auto_grow_and_tokenize.py` (an unattended driver, written after this exploratory attempt) grew whichever lane fell short of its absolute token target and re-ran the pipeline until both were met. The result: 635,204,333 final tokens (167,787,251 manual $\geq$ 100M target, 467,417,082 downloaded $\geq$ 400M target), a BPE tokenizer at vocab_size=10,000, and 0

shared documents against Hindi (see "Cross-Language Check" in `phase1_report_nepali.pdf`). `scripts/validate_phase1_nepali.py` passes every check. **Nepali is now the final, graded Model L.**

`bhojpuri/` and this report's Bhojpuri content are kept in the repository exactly as they were – nothing deleted, nothing re-run – as a complete, working fallback and a record of that earlier work. The Nepali pipeline's code (`nepali/scripts/`, `nepali/configs/`) is versioned the same way Hindi's and Bhojpuri's own pipeline code is; its data (raw/interim/final, ∼20GB) is not versioned, for the same "code in git, corpus data elsewhere" reason Hindi and Bhojpuri's own raw/cleaned corpora are not versioned either.

Sources and References

1. Hindi Wikipedia XML dump (self-extracted via WikiExtractor). Source: Wikimedia dumps, https://dumps.wikimedia.org.
2. Bhojpuri Wikipedia XML dump (self-extracted via WikiExtractor). Source: Wikimedia dumps, https://dumps.wikimedia.org.
3. Gadya Kosh (Hindi prose wiki). https://gadyakosh.org. Author-copyright content, collected for coursework research use.
4. Kavita Kosh (Hindi/Urdu poetry wiki, also hosts Bhojpuri/Awadhi/ Maithili sections). https://kavitakosh.org. Author-copyright content, collected for coursework research use.
5. Patrika (Hindi newspaper archive, Rajasthan Patrika group). https://www.patrika.com. Collected for coursework research use.
6. Khabar Bhojpuri (Bhojpuri news site). https://khabarbhojpuri.com.
7. Anjoria (Bhojpuri content site). https://anjoria.com.
8. Jogira (Bhojpuri literature/culture site). https://jogira.com.
9. Sirijan (Bhojpuri quarterly e-magazine). https://sirijan.com.
10. Internet Archive (OCR/text extraction of books, journals, government debate transcripts). https://archive.org.
11. HuggingFaceFW/fineweb-2 (Hindi and Bhojpuri partitions). https://huggingface.co/datasets/HuggingFaceFW/fineweb-2.
12. AI4Bharat Sangraha (verified and unverified Hindi partitions). https://huggingface.co/datasets/ai4bharat/sangraha.
13. Satyam810/BhojpuriCorpus (17 partitions: fish_food, jnu_bhltr, hplt, wikipedia, eng_bho, bhojpuri_lr_v3, kumar_bhojpuri, iit_bhu, fleurs_bhojpuri, google_smol, glotcc, rural_women, ud_bhtb, and others). https://huggingface.co/datasets/Satyam810/BhojpuriCorpus.
14. BHLTR: Bhojpuri-Hindi Language Technology Resources (JNU), monolingual and parallel corpora. https://github.com/shashwatup9k/BHLTR, CC-BY-NC-SA-4.0.

15. `wikimedia/wikipedia:20231101.hi` – evaluated, quarantined, not used (redundant with the self-extracted Hindi Wikipedia dump above). https://huggingface.co/datasets/wikimedia/wikipedia.

16. CulturaX, sil-ai/bloom-lm, AI4Bharat Sangraha (Bhojpuri rows), Glot500, MADLAD-400, CC-100 – evaluated for Bhojpuri and excluded; see Section 3 for reasons per source.

17. SentencePiece (tokenizer library). Kudo, T. and Richardson, J. (2018). *SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing.* EMNLP 2018 (System Demonstrations). https://github.com/google/sentencepiece.

18. `langdetect` (Python language-identification library used as one signal in the language filter, with known no-Bhojpuri-label limitation documented throughout Section 3). https://pypi.org/project/langdetect/.